

First Course Handout

AE646: Scientific Machine Learning for Fluid Mechanics

Semester 2026–27-I

Course code and title	AE646: Scientific Machine Learning for Fluid Mechanics
L–T–P–D–C	2–0–3–0–9
Lecture slot	Wednesday and Friday, 9:00–10:15 AM
Venue	Classroom (2nd floor), National Wind Tunnel Facility Building
Instructor	Dr. Sathesh Mariappan
Designation	Professor, Department of Aerospace Engineering, IIT Kanpur
Department	Aerospace Engineering, IIT Kanpur
Office	210A, National Wind Tunnel Facility Building
Email	sathesh@iitk.ac.in
Phone	+91-5122596331
Preferred mode of contact	Email first to fix an appointment
Mode of announcements	HelloIITK portal: https://hello.iitk.ac.in/ / email, as applicable
Teaching Assistants	Vishal Choudhari (vuchoudhari25@iitk.ac.in); Tanish Shetty (tanishs25@iitk.ac.in)

Expected background: Fluid Mechanics, CFD, Mathematics, and Python programming are helpful for this course.

1. Course objective

This course introduces scientific machine learning methods for fluid mechanics and related aerospace applications. The course emphasizes hands-on learning through computational projects, where students will build, train, evaluate, and interpret machine-learning models for physical systems.

The focus is not only on using neural networks, but also on understanding their limitations, comparing them with classical baselines, and evaluating whether the results are physically meaningful.

2. Learning outcomes

At the end of the course, students should be able to:

1. Represent fluid-mechanics data as fields, snapshots, time series, and machine-learning-ready tensors.
2. Apply SVD, PCA, POD, and reduced-order modeling ideas to flow data.
3. Train basic neural-network models for regression, reconstruction, and prediction.
4. Understand optimization, validation loss, overfitting, regularization, and model selection.
5. Implement physics-informed neural networks for simple PDEs.
6. Understand operator-learning methods such as DeepONet and Fourier Neural Operators.
7. Use CNNs, autoencoders, and recurrent models for flow-field reconstruction and temporal prediction.
8. Compare SciML models against suitable baselines.
9. Evaluate models using quantitative metrics and physical diagnostics.
10. Present reproducible computational results with clear physical interpretation.

3. Course format

The course is primarily hands-on and project-based. There will be no conventional written mid-semester or end-semester examination. Instead, project evaluations will replace these components.

Students will work on a semester-long project, evaluated in three stages:

1. Stage 1: Project proposal
2. Stage 2: Implementation and interim results
3. Stage 3: Final report, code, presentation, and viva

Students may choose from a defined project bank or propose a related topic with instructor approval. Project groups may have up to four students.

4. Tentative lecture plan

The IIT Kanpur Academic Calendar 2026 lists classes for 2026–27-I from July 30 to November 13, 2026, with mid-semester examinations during September 13–20, mid-semester recess during October 18–25, and end-semester examinations during November 16–26. It also notes that October 10, 2026 follows the Friday timetable, and that during the Gymkhana Holiday week, the Friday will be a holiday while the Wednesday timetable will be followed on Thursday.

Lecture	Topic / activity
1	Introduction to Scientific Machine Learning for Fluid Mechanics
2	Flow data, fields, snapshots, notation
3	Linear algebra review: vectors, matrices, norms, projections
4	Regression, least squares, train/validation/test split
5	Approximation, projection error, model complexity
6	SVD, PCA, POD, and reduced-order representation
7	Probability and statistics review
8	Shallow neural networks and universal approximation
9	Optimization, validation loss, model selection
10	Backpropagation, automatic differentiation, regularization
11	TensorFlow/PyTorch implementation workflow
–	Stage 2 interim evaluation before the mid-semester examination period
–	Mid-semester examination period - no AE646 activity
12	PINN basic idea
13	Burgers equation PINN
14	DeepONet and operator learning
–	Holiday / buffer
15	Fourier Neural Operator
16	CNNs for field data - Friday timetable compensation class
17	Encoders, decoders, and autoencoders
18	Physics-aware autoencoders
–	Mid-semester recess
19	Latent-space dynamics
20	RNN, LSTM, GRU temporal prediction
21	Physics-aware temporal prediction and stable rollouts
22	Model selection, comparison, and closing remarks
–	Stage 3 final project evaluation week
–	Final report and code submission deadline
–	End-semester examination period - no AE646 activity

Note: The Gymkhana Holiday week compensation will be adjusted once the exact holiday date is announced.

5. Project themes

Students may choose from a defined project bank or propose a related project. Project groups may have up to four students. All projects should have similar expected difficulty and must include:

1. A physical/scientific problem
2. A baseline method
3. One SciML model
4. Train/validation/test split
5. Quantitative evaluation
6. Physical interpretation
7. Reproducible code

Possible project themes include heat-flux reconstruction from temperature fields, temperature-field to velocity-field reconstruction, flame-image classification for combustion dynamics, flame-image based prediction of pressure bursts, POD vs autoencoder reconstruction, latent-space dynamics, RNN/LSTM/GRU prediction of POD coefficients, PINNs, DeepONet/FNO operator learning, wake dynamics prediction, boundary-layer transition detection from thermal images, and sparse sensor reconstruction of flow or thermal fields.

Students may also propose projects related to experimental diagnostics, combustion instability, aerodynamic flow reconstruction, thermal imaging, reduced-order modeling, or PDE learning. Students may also undertake a project related to their thesis work, provided it is connected to the course content and approved by the instructor.

5.1 Suggested synthetic-data project themes and repositories

Students may use simple Python-generated datasets, instructor-provided data, or approved open-source repositories. The repositories and data sources listed below are suggestive and not exhaustive. Students may choose other suitable repositories, datasets, or self-generated synthetic data, provided the scope remains comparable and the choice is approved. All external repositories, datasets, codes, tutorials, and papers used in the project must be properly cited and acknowledged in the report and presentation. Each project must still include a baseline, one SciML model, quantitative evaluation, physical interpretation, and reproducible code.

No.	Project theme	Suggested repository / data source	Suitable baseline	Suggested SciML model
1	Burgers equation surrogate modeling	CFD Python; PDEBench	Finite-difference solver / low-resolution solver	PINN, DeepONet, FNO
2	Heat equation / thermal diffusion inverse problem	CFD Python-style custom solver; PDEBench	Finite-difference inverse estimate	CNN, U-Net, PINN
3	Advection-diffusion scalar transport	PhiFlow; PDEBench; custom solver	Numerical solver / POD regression	FNO, DeepONet, CNN predictor
4	Lid-driven cavity flow reconstruction	CFD Python; CFDBench; OpenFOAM	POD reconstruction / interpolation	CNN autoencoder, U-Net
5	Cylinder wake / Karman vortex street prediction	CFDBench; OpenFOAM; SU2	POD + linear dynamics	Autoencoder + LSTM/GRU, ConvLSTM
6	POD vs autoencoder compression of flow fields	Synthetic vortex/blob fields; CFDBench; PDEBench	POD	Convolutional autoencoder
7	Sparse sensor reconstruction of flow/thermal fields	Any generated flow/thermal dataset; PDEBench; PhiFlow	Gappy POD / interpolation	Sensor-to-field MLP, DeepONet
8	Operator learning for parametric PDEs	PDEBench; NeuralOperator examples; original FNO repository	MLP / low-resolution solver	DeepONet, FNO
9	Boundary-layer or transition-like thermal image detection	Custom synthetic thermal fields; SU2/OpenFOAM-generated data	Gradient/threshold method	CNN segmentation, U-Net
10	Synthetic flame-image classification	Custom generated flame-like image sequences	Intensity/area thresholding	CNN, CNN-LSTM

No.	Project theme	Suggested repository / data source	Suitable baseline	Suggested SciML model
11	Synthetic thermoacoustic oscillator modeling	Custom nonlinear oscillator data	FFT/Hilbert-envelope analysis	LSTM, GRU, Neural ODE
12	Parametric aerodynamic surrogate modeling	SU2; OpenFOAM; CFDBench-style data	Polynomial regression / POD regression	MLP, DeepONet, FNO

Repository notes. CFD Python is suitable for beginner-friendly PDE data generation and basic CFD solvers such as convection, diffusion, Burgers equation, cavity flow, and simple Navier–Stokes exercises. PDEBench provides benchmark datasets and code for scientific machine learning on PDEs. NeuralOperator provides PyTorch implementations of neural-operator models including FNO. PhiFlow is useful for differentiable PDE/fluid simulations. CFDBench is useful for CFD surrogate and generalization studies. OpenFOAM and SU2 are better suited for advanced groups who want to generate their own CFD data.

6. Grading policy

The course will be evaluated entirely through projects.

Stage	Component	Weight
Stage 1	Project proposal	20%
Stage 2	Implementation and interim results	35%
Stage 3	Final report, code, presentation, and viva	45%
Total		100%

There will be no separate written mid-semester or end-semester examination. Stage 2 and Stage 3 will be completed before the official examination periods.

7. Report weightage and LLM-aware evaluation

Since written reports can be assisted or generated by LLMs, report marks will be limited. The main evidence of learning will be implementation, reproducibility, results, interpretation, live explanation, and viva.

Report component	Marks
Stage 1 proposal report	12
Stage 2 interim report	6
Stage 3 final report	10
Total report-based marks	28/100

The remaining 72 marks will be based on implementation, results, code, live explanation, presentation, and viva.

8. Stage-wise evaluation details

8.1 Stage 1: Project proposal - 20 marks

Tentative due date: Aug 29, 2026

Evaluated by: Teaching Assistants, with instructor moderation if required

Expected submission: 2–3 page proposal.

Criterion	Marks
Problem clarity and relevance to AE646	4
Dataset/PDE/experimental source and feasibility	4
Baseline method and proposed SciML method	4
Evaluation metrics and expected results	4
Short TA interaction / ownership check	4
Total	20

The proposal should include project title and team members, problem statement, dataset/PDE/experimental source, baseline method, proposed SciML method, evaluation metrics, expected figures/tables, work plan, and AI-tool-use declaration.

8.2 Stage 2: Implementation and interim results - 35 marks

Material due: Sep 8, 2026

Evaluation dates: Sep 9 and Sep 11, 2026

Criterion	Marks
Dataset/PDE setup and preprocessing	5
Baseline implementation	7
Initial SciML implementation	7
Preliminary quantitative results	5
Code clarity and reproducibility	5
Interim report	6
Live code explanation / viva	5
Total	35

Expected submission: 4–5 page interim report, working code or Colab notebook, dataset generation/loading completed, baseline implemented, initial SciML model attempted, preliminary plots/tables, and issues faced with next-step plan. At this stage, the baseline should be complete and the main SciML model should be at least partially working.

8.3 Stage 3: Final report, code, presentation, and viva - 45 marks

Evaluation week: Nov 9–13, 2026

Final report/code due: Nov 13, 2026

Criterion	Marks
Correctness and completeness of implementation	8
Baseline vs SciML comparison	7
Quantitative evaluation and metrics	7
Physical interpretation of results	7
Code reproducibility	4
Final report	10
Presentation	2
Individual viva / contribution check	5
Total	45

Expected submission: final report around 8–10 pages, final code/Colab notebook, final presentation, individual contribution statement, AI-tool-use declaration, and viva/code walkthrough.

9. Submission format and report page limits

All submissions must be made in the announced course channel. Unless specified otherwise, each submission should contain a PDF report and the corresponding code/notebook files. The code submission

should include sufficient instructions to reproduce the main results.

Stage	Report length	Submission items
Stage 1	2–3 pages	Proposal PDF containing the problem statement, dataset/PDE/source, baseline, proposed SciML method, evaluation metrics, expected outputs, work plan, and AI-tool-use declaration.
Stage 2	4–5 pages	Interim report PDF, working code/Colab notebook, dataset generation/loading instructions, baseline results, initial SciML results, preliminary plots/tables, and issues/next steps.
Stage 3	8–10 pages	Final report PDF, final code/Colab notebook, README or run instructions, final plots/tables, individual contribution statement, AI-tool-use declaration, and presentation file.

The report page limits are indicative upper limits for the main text. References and compact appendices may be used when needed, but the core discussion should remain concise and focused on the scientific workflow: problem setup, baseline, SciML model, evaluation, interpretation, limitations, and reproducibility.

10. Tentative submission schedule

Date	Activity
Aug 14	Project bank released
Aug 21	Team formation deadline
Aug 29	Stage 1 proposal submission
Sep 5	Stage 1 feedback returned
Sep 8	Stage 2 interim material due
Sep 9	Stage 2 review, slot 1
Sep 11	Stage 2 review, slot 2
Oct 16	Optional ungraded progress checkpoint
Oct 30	Final result draft / preliminary plots due for feedback
Nov 6	Lecture portion completed
Nov 9–13	Stage 3 final project presentations, viva, and code review
Nov 13	Final report and code submission deadline

11. Attendance policy

Attendance is recommended. Students are strongly encouraged to attend lectures, project discussions, and evaluation-related sessions because the course is hands-on and project-based.

Students are responsible for all announcements, deadlines, project instructions, and evaluation requirements communicated in class or through official course communication channels.

12. Academic integrity and AI-tool policy

Students may use AI tools such as ChatGPT, Gemini, Claude, Copilot, or similar systems for understanding concepts, coding assistance, debugging, grammar correction, idea refinement, and plotting help. However, the submitted work must be understood, verified, and owned by the students.

Students must be able to explain every part of their submitted report, code, figures, results, and conclusions during evaluation. Inability to explain submitted work may lead to loss of marks, irrespective of the quality of the written submission.

Permitted

- Discussing concepts with classmates
- Using open-source libraries with proper acknowledgment
- Using AI tools with declaration
- Getting help in debugging or formatting
- Referring to published papers, tutorials, and documentation with citation

Not permitted

- Submitting work copied from another group
- Submitting AI-generated code/report without understanding
- Fabricating results
- Reusing figures/results without citation
- Claiming contribution to work not performed
- Misrepresenting datasets, model performance, or experimental outcomes

Any academic integrity violation may lead to mark deduction, zero marks in the concerned component, and/or reporting to the appropriate institute body, depending on severity. If widespread unfair means are observed, the instructor may revise the evaluation procedure while remaining consistent with institute norms.

13. Makeup and submission policy

Project deadlines and evaluation dates are listed in the tentative submission schedule above. Students are expected to follow these announced deadlines and attend the scheduled evaluation sessions.

Absence from Stage 2 or Stage 3 evaluation requires prior approval or valid documentation in case of medical, institutional, or emergency reasons.

The instructor's decision will be final for cases not covered above.

14. Software and computational requirements

Students are expected to use Python-based scientific computing tools. Suggested tools include Python, NumPy, SciPy, Matplotlib, PyTorch or TensorFlow, Google Colab / local Python environment, Jupyter notebooks, and Git/GitHub or equivalent version control. Students should ensure that their final code is reproducible and clearly documented.

15. References and reading material

Suggested references:

1. Brunton, S. L. and Kutz, J. N., *Data-Driven Science and Engineering*.
2. Goodfellow, I., Bengio, Y., and Courville, A., *Deep Learning*.
3. Raissi, M., Perdikaris, P., and Karniadakis, G. E., physics-informed neural networks papers.
4. Lu et al., DeepONet papers.
5. Li et al., Fourier Neural Operator papers.
6. Course lecture notes and Colab notebooks provided during the semester.

Additional references will be provided topic-wise.

Useful repository links

- CFD Python: <https://github.com/barbagroup/CFDPython>
- PDEBench: <https://github.com/pdebench/PDEBench>
- NeuralOperator: <https://github.com/neuraloperator/neuraloperator>
- Original FNO repository: https://github.com/scaomath/fourier_neural_operator
- PhiFlow: <https://github.com/tum-pbs/phiflow>
- CFDBench: <https://github.com/luo-yining/CFDBench>
- OpenFOAM: <https://github.com/OpenFOAM/OpenFOAM-12>
- SU2: <https://su2code.github.io/>